Binary Heap

CSE 4303: Data Structures
Lecture 9

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

January 22, 2026

Recap: The Need for a Better Priority Queue

1. What is a Priority Queue?

- 👉 An abstract data type where every element has a *priority*.
- 👉 Elements with higher priority are served before elements with lower priority.
- 👉 Key Operations: `insert(item)` and `extract_max()` / `extract_min()`.

2. The Problem with Naive Implementations

If we implement a Priority Queue using simple Arrays or Linked Lists, we hit a performance bottleneck.

Approach	**Insert**	**Extract Max**
Unsorted Array	$O(1)$	$O(N)$ *(Too slow to find)*
Sorted Array	$O(N)$ *(Too slow to add)*	$O(1)$
Binary Heap	$\mathbf{O(\log N)}$	$\mathbf{O(\log N)}$

Conclusion: Arrays are either too slow to insert or too slow to search. We need a structure that balances both operations efficiently. **Enter the Binary Heap.**

What is a Binary Heap?

A Binary Heap is a binary tree with two specific constraints:

Heap Property (Max-Heap):

- 👉 For every node i, the value of the parent is greater than or equal to the values of its children.
- 👉 $\text{Parent}(i) \geq \text{Children}(i)$
- 👉 *Result: The largest element is always at the root.*

Shape Property: A *Complete Binary Tree*.

What is a Binary Heap?

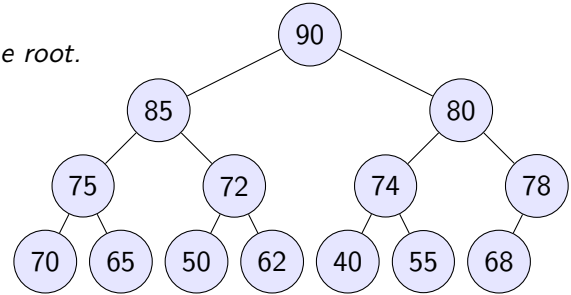
A Binary Heap is a binary tree with two specific constraints:

Heap Property (Max-Heap):

- For every node i, the value of the parent is greater than or equal to the values of its children.
- $\text{Parent}(i) \geq \text{Children}(i)$
- *Result: The largest element is always at the root.*

Shape Property: A *Complete Binary Tree*.

- **Recap** All levels are fully filled, except possibly the last.
- **Recap** Nodes are filled from **left to right**.



Max Heap

Implementation: Array vs. Linked List

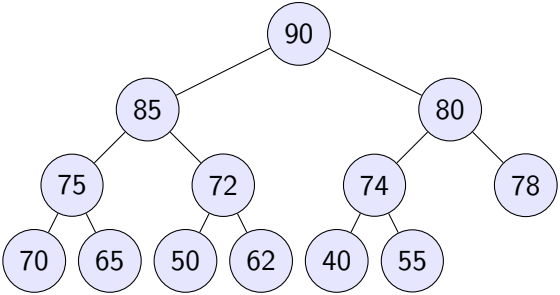
Option 1: Linked Nodes

- Uses pointers: `left`, `right`, `parent`.
- **The Problem:** Heaps must remain *Complete*.
- Finding the next free spot (bottom-left-most available leaf) requires a Level Order Traversal ($O(N)$) or complex tracking.
- **Verdict:** Inefficient for Heaps.

Option 2: Array (The Standard)

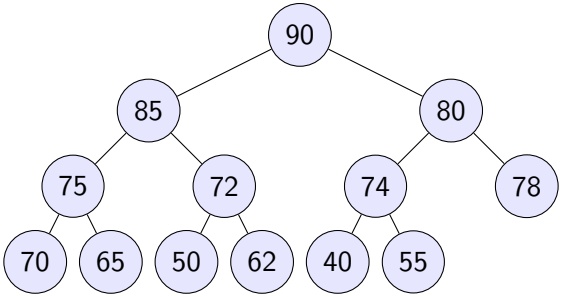
- Stores nodes in *Level Order*.
- No pointers needed (Implicit structure).
- **The Advantage:** The next free spot is always at `index = size`.
- **Verdict:** $O(1)$ access to insertion point.

Mapping Levels to Indices (0-based Indexing)



1	2	3	4	5	6	7	8	9	10	11	12	13
90	85	80	75	72	74	78	70	65	50	62	40	55	*empty...*

Mapping Levels to Indices (0-based Indexing)



1	2	3	4	5	6	7	8	9	10	11	12	13
90	85	80	75	72	74	78	70	65	50	62	40	55	*empty...*

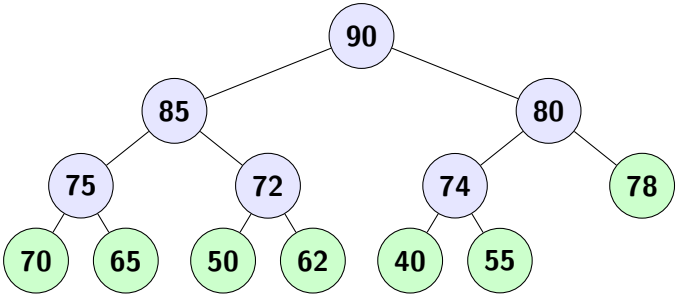
Navigation Formulas (for index i)

Left Child: $2i$ | Right Child: $2i + 1$ | Parent: $\lfloor \frac{i}{2} \rfloor$

The Leaf Node Property

Optimization Insight: In a heap of size $N = 13$, notice where the leaves start.

- **Internal Nodes:** Indices 1 to $\lfloor N/2 \rfloor$ (1 to 6).
- **Leaf Nodes:** Indices $\lfloor N/2 \rfloor + 1$ to N (7 to 13).



1	2	3	4	5	6	7	8	9	10	11	12	13
90	85	80	75	72	74	78	70	65	50	62	40	55

Internal Nodes $(1 \ldots 6)$

Leaf Nodes $(7 \ldots 13)$

Fixing a Violation: Heapify Down I

The Scenario: A node (Parent) violates the Max-Heap property because it is **smaller** than one or both of its children.

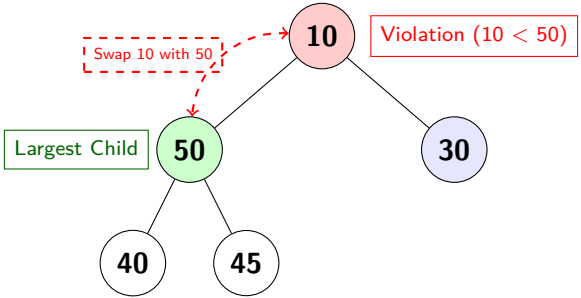
The Fix (Percolate Down): We must "demote" the node down the tree until it finds its correct spot.

The "Swap with Largest" Rule

To maintain the structure, you must swap the parent with the **LARGEST** of its two children.

- 👉 Compare $A[i]$, $A[Left]$, and $A[Right]$.
- 👉 If $A[i]$ is largest $\rightarrow$ Done.
- 👉 Else $\rightarrow$ Swap $A[i]$ with the largest child and repeat.

Fixing a Violation: Heapify Down II



Why specifically the largest? If we swapped 10 with 30, then 30 would be above 50, which is still a violation!

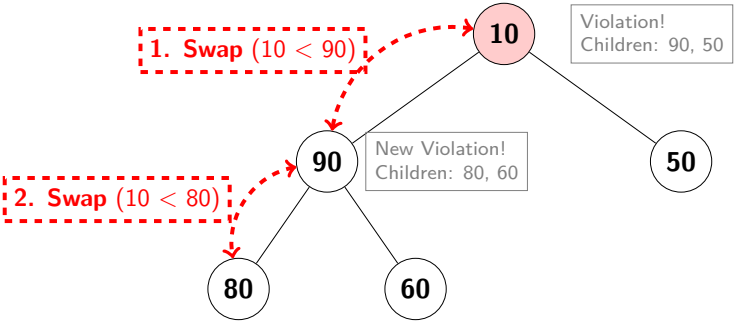
Heapify Down: The Cascade Effect I

The problem moves down:
Swapping a violating node with its child fixes the local problem, but it might break the Heap Property at the *new* position.

Pseudocode: `max_heapify(i)`

```
l = 2*i;
r = 2*i + 1;
if l ≤ heapSize and heap[l] > heap[i]
  largest = l;
else
  largest = i;
if r ≤ heapSize and heap[r] > heap[largest]
  largest = r;
if largest != i
  exchange heap[i] ↔ heap[largest];
  max_heapify(largest);
```

Heapify Down: The Cascade Effect II



Time Complexity: O(log N) - Proportional to height

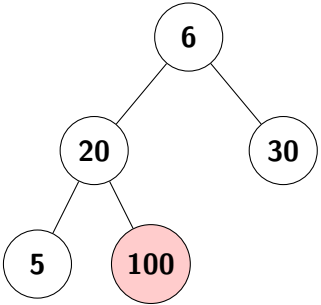
Why Top-Down Build Fails I

The Naive Idea: Iterate from $1 \to N$ calling `max_heapify`.

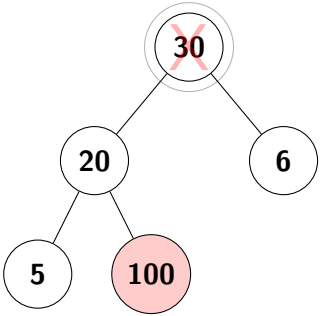
- **Why it fails:** `max_heapify(i)` assumes the subtrees of i are already valid heaps.
- If we start at the root, the bottom is unordered. Large values at the bottom (like **100** below) never get a chance to bubble up.

Why Top-Down Build Fails II

Initial State



After max_heapify(Root)



Why Top-Down Build Fails III

The Failure

We ran `max_heapify` on the Root. It compared $6, 20, 30$ and swapped $6 \leftrightarrow 30$. **The algorithm terminates. Result:** Root is 30, but the max is 100. The heap property is broken.

The Solution: Bottom-Up Build Heap

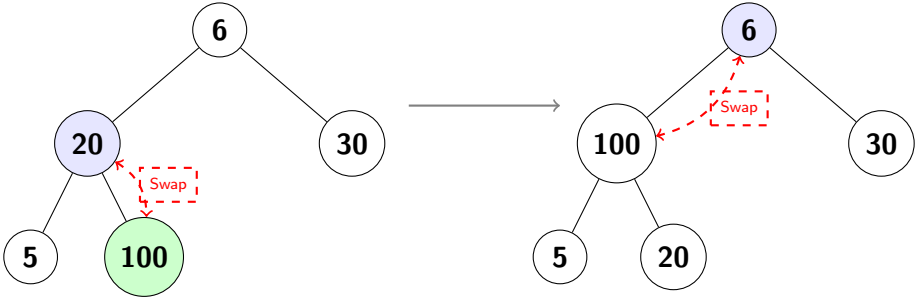
Strategy: Fix the small sub-trees first, then move up.

- 👉 **Start:** Index $\lfloor N/2 \rfloor$ (The last internal node).
- 👉 **Direction:** Iterate backwards down to 1.
- 👉 **Why it works:** By the time we reach node i, we guarantee that `left(i)` and `right(i)` are already valid heaps.

The Algorithm

```
for i = ⌊heapSize / 2⌋ down to 1:
  max_heapify(heap, i, heapSize)
```

Bottom-Up Build Heap



Result: The "Giant" (100) successfully moves from the bottom leaf to the Root!

Analysis: Build Heap Complexity (Intuition)

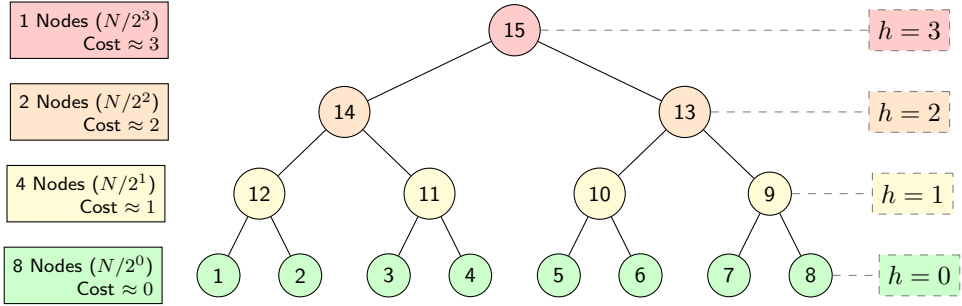
The Naive Analysis ($O(N \log N)$):

- We call `max_heapify` for $N/2$ nodes.
- Each call takes $O(\log N)$.
- Total $\approx N/2 \times \log N = O(N \log N)$. **(This is an upper bound, but not tight!)**

The Better Analysis ($O(N)$):

- `max_heapify` takes time proportional to the **height** of the node, not always $\log N$.
- **Observation:**
  - The leaves (50% of nodes) are at height 0. **Work = 0**.
  - The next level (25% of nodes) is at height 1. **Work = 1 step**.
  - Only the Root (1 node) needs height $h = \log N$.
- Most nodes have small heights!

Visualizing Height for Complexity Analysis I



Notice: As the cost (h) increases, the number of nodes paying that cost decreases exponentially.

Analysis: Build Heap (Mathematical Proof)

Total Work = Sum of heights of all nodes.
For a heap of height H, the maximum number of nodes at height h is $\lceil \frac{N}{2^{h+1}} \rceil$.

$$S = \sum_{h=0}^{\lfloor \lg N \rfloor} \frac{N}{2^{h+1}} \cdot O(h) = N \sum_{h=0}^{\lfloor \lg N \rfloor} \frac{h}{2^{h+1}}$$

Recognize the series $\sum_{h=0}^{\infty} \frac{h}{2^h}$. This is an Arithmetico-Geometric Series that converges to **2**.

$$S \approx N \cdot \left(\frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \dots \right) \leq N \cdot 2 = O(N)$$

Conclusion

Building a heap from an unordered array takes **Linear Time** $O(N)$, not $O(N \log N)$. This is very efficient!

Insertion (Percolate Up) I

Algorithm:

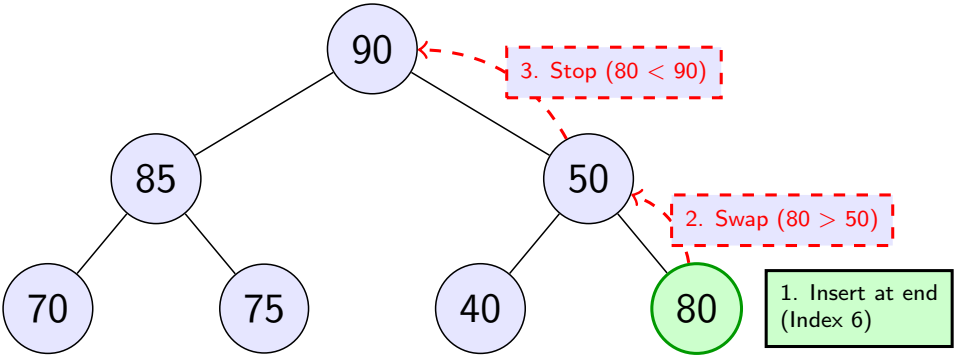
1. **Shape First:** Insert the new key at the end of the array (next available leaf).
2. **Order Second:** Compare the new key with its **Parent**.
3. **Swap** if the Child $>$ Parent. Repeat until the heap property is restored.

"The new guy has to earn his way to the top."

Complexity

- 👉 **Time:** $O(\log N)$ (Height of tree)
- 👉 **Space:** $O(1)$

Insertion (Percolate Up) II




```
parent_index = current_index / 2;
```

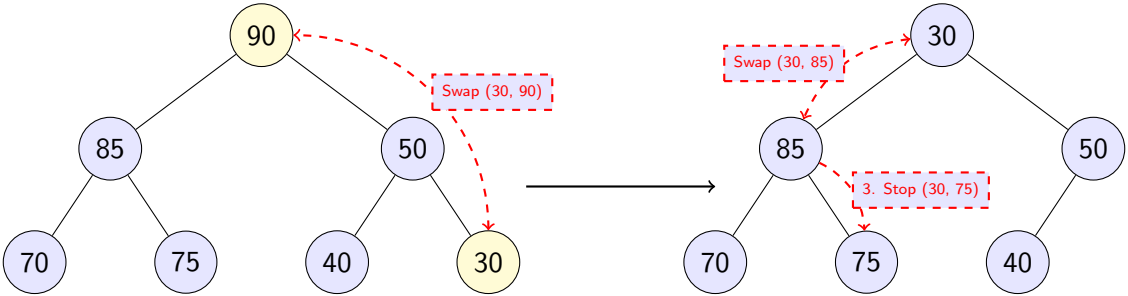
Extract Max (Delete Root) I

The Goal: Remove and return the maximum element (the Root).
The Challenge: Removing the root leaves two disconnected sub-trees. We must preserve the *Shape Property* (complete binary tree).

Algorithm ($O(\log N)$)

1. **Swap:** Exchange the Root with the **Last Leaf** (last index).
2. **Remove:** Decrement `heap_size` (effectively removing the Max).
3. **Fix:** The new Root is likely small. Call `max_heapify(1)` (Bubble Down).

Extract Max (Delete Root) II



By moving the last element to the top, we keep the tree "Complete". Then we just fix the order.

Arbitrary Deletion: The Search Problem

Task: Delete a node with value X (not necessarily the root).

Step 1: Find the node (The Bottleneck!)

- 👉 Heaps are **weakly ordered**. Left child isn't necessarily smaller than Right child.
- 👉 We cannot use Binary Search. We must scan the array linearly.
- 👉 **Search Cost:** $O(N)$.

Step 2: Delete at Index i (If we already know the index)

- 👉 Swap $A[i]$ with $A[size]$.
- 👉 Decrease size.
- 👉 If new $A[i] > Parent$: `Percolate Up`.
- 👉 Else: `max_heapify(i)` (Down).
- 👉 **Delete Cost:** $O(\log N)$.

Verdict

`delete(value)` is $O(N)$ because finding the value takes $O(N)$.
`delete(index)` is $O(\log N)$ if we have a handle to the index.

Modifying Keys: Increase & Decrease

Sometimes priorities change dynamically (e.g., in Dijkstra's Algorithm).

1. **Increase Key** (Value goes UP)
   - 👉 Find the value $O(N)$ (like arbitrary delete)
   - 👉 Node becomes larger than children (safe), but might be larger than **Parent**.
   - 👉 **Action:** Update value, then **Percolate Up** for max heap.
2. **Decrease Key** (Value goes DOWN)
   - 👉 Find the value $O(N)$ (like arbitrary delete)
   - 👉 Node becomes smaller than parent (safe), but might be smaller than **Children**.
   - 👉 **Action:** Update value, then **Heapify Down** for max heap.

Time Complexity for Both: If value index is given $O(\log N)$ (bounded by tree height). Otherwise Find $O(N)$ + Update $O(\log N)$.

Application: Heap Sort

Idea: If we can extract the maximum element in $O(\log N)$, Do it $N-1$ times to get it sorted.
Algorithm:

1. **Build Max Heap** from unordered array. $O(N)$
2. **Loop** i from N down to 2: $O(N \times \log N)$
   - Swap Root ($A[1]$) with Last ($A[i]$).
   - The largest element is now "sorted" at the end.
   - Reduce heap size by 1.
   - Call `max_heapify(1)` to fix the root.

Complexity

- **Time:** $O(N \log N)$ (Best, Average, and Worst case).
- **Space:** $O(1)$ (In-place sorting).

Priority Queue Implementation

A Binary Heap is the standard implementation for a Priority Queue.

PQ Operation	Heap Equivalent	Complexity
`peek_max()`	Return `A[1]` (indexed 1) (Root)	$O(1)$
`insert(k)`	1. Add to end (`A[++size]`) 2. **Percolate Up**	$O(\log N)$
`extract_max()`	1. Save Root, Swap Root $\leftrightarrow$ Last 2. `size = size - 1` 3. `max_heapify(1)` (Percolate Down)	$O(\log N)$

Note on Min-Heaps: All logic is identical, but inequalities are reversed (Parent $\leq$ Child).

🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 12**:Binary Heaps

Thank You!

👋 Thank you for your time and attention. 👋

🗨️ Any questions or clarifications. 💭